CS224 - Fall 2025 - Lab #1 (Version 01: October 03, 12:05)

MIPS Assembly Language Programming: Arrays, Bit Manipulation, and Subprograms

Dates: (TAs)

Section 1: Wed, 15 Oct, 13:30-17:20 (Kadri, Mert, Omid, Yağız)
Section 2: Thu, 16 Oct, 13:30-17:20 (Berkan, Kadri, Mahyar)
Section 3: Thu, 16 Oct, 8:30-12:20 (Mert, Tuna, Yağız)
Section 4: Fri, 17 Oct, 13:30-17:20 (Mahyar, Omid, Tuna)

TA Full Name (email address: @bilkent.edu.tr) No of Labs

Berkan Şahin: (berkan.sahin) 1

Kadri Gofralılar: (kadri.gofralilar) 2

Mahyar Fardinfar (mahyar.fardinfar) 2

Mert Barkın Er (barkin.er) 2

Omid Feizi: (omid.feizi) 2

Sepehr Bakhshi: (sepehr.bakhshi) *

Tuna Okçu (tuna.okcu) 2

Yağız Özkarahan (yagiz.ozkarahan) 2
* Coordinating TA.

Summary

Purpose: Understanding array access, dynamic array allocation, passing arguments to and receiving results from subprograms, understanding bit operations.

Program Descriptions

Preliminary Work (50 points)

1. **ArrayOperations**: Print Array, Find Min Max, …
2. **FindOccurrences**: Find number of occurrences of the values 0 to 9 in an array.

Lab Work (50 points)

1. **ExchangeRegBytes**: Exchange rightmost two bytes and rightmost two bytes of a register.
2. **ReverseArray**: Invert array (first entry becomes last, last entry becomes first etc.).

Implementation Notes and Requirements for Subprograms

1. In this lab the use of $s registers in subprograms is not mandatory: You may use $t registers in the subprograms to simplify the programs. In Lab 2 you will be required to use $s registers (after saving them to stack) in the subprograms and not allowed to use $t registers in them.
2. For passing arguments to subprograms use argument ($a) registers and for returning results to the caller use $v0 and $v1 registers.

3. You can only use a limited number of pseudo instructions in your lab work. They are the following: lw and sw instructions (like: lw $t0, a; sw $t0, a), load address (like: la $t0, array), and conditional branch instructions (like: bgt, blt ...), and load immediate for readability (like: li $v0, 10).
4. All variable defined in the data segment is accessible by all subprograms. However to use the variables defined in the data segment you have to pass them to subprograms using argument registers.

Important Notes for All Labs About Attendance, and Performing-Presenting Your Work

You are obliged to read this document word by word and are responsible for the mistakes you make by not following the rules.

1. Not attending to the lab means 0 out of 100 for that lab. If you attend the lab but do not submit the preliminary part you will lose only the points for the preliminary part.
2. Try to complete the lab part at home . Make sure that you show your work to your TAs and answer their questions to show that you know what you are doing before uploading your lab work and follow the instructions of your TAs.
3. In all labs if you are not told you may assume that inputs are correct. This is for simplicity.
4. In all labs when needed you have to provide a simple user interface for inputs and outputs.
5. Format of your submitted work
   You have to provide a neat presentation prepared in txt form. Your programs must be easy to understand and well structured.
   Provide following six lines at the top of your submission for preliminary and lab work (make sure that you include the course no. CS224, important for ABET documentation).

   CS224
   Lab No.
   Section No.
   Your Full Name
   Bilkent ID
   Date

   Please also make sure that your work is identifiable: In terms of which program corresponds to which part of the lab.
6. **Make sure that the code you submit is really yours and has been internalized. You may experiment with chatGPT-like tools but your work should be yours. You are not allowed to use chatGPT in your final submitted work. Too much help from GenAI tools will disadvantage you in the exams.**

   Note that MOSS is capable of detecting ChatGPT code. Furthermore, remember that, the code you use from ChatGPT can also be used by another student in the course.

   The purpose of education is learning to learn. Learn how to learn. Use the opportunity provided in

this course. It is very probable that in your life you will do things different from what you have learned during your undergraduate education.

DUE DATE FOR PRELIMINARY WORK: SAME FOR ALL SECTIONS

NO late submission will be accepted. Please do not try to break this rule and any other rule we set.

a. Please upload your programs of preliminary work to Moodle by 13:30 on Wednesday, 15 October, 2025. Please note that the submission closes sharp at 13:30 and no late submissions will be accepted

b. You can make resubmissions so do not wait for the last moment. Submit your work earlier and change your submitted work if necessary. Note that only the last submission will be graded.

c. Please familiarize yourself with the Moodle course interface, find the submission entry early, and avoid sending an email like "I cannot see the submission interface." The submission interface will be available soon.

d. Do not send your work by email attachment: they will not be processed. They have to be in the Moodle system to be processed.

e. Use filename **StudentID_FirstName_LastName_SecNo_PRELIM_LabNo.txt** Only a NOTEPAD FILE (txt file) is accepted. Any other form of submission receives 0 (zero).

DUE DATE FOR LAB WORK: YOUR LAB DAY (different for each section)

a. You have to demonstrate your lab work to your TA for grading. Do this by **12:00** in the morning lab and by **17:00** in the afternoon lab. Your TAs may give further instructions on this. If you wait idly and show your work last minute, your work may not be graded.

b. At the conclusion of the demo for getting your grade, you will **upload your Lab Work** to the Moodle Assignment, for MOSS. See below for the details of lab work submission.

c. Try to finish all of your lab work before coming to the lab, but make sure that you upload your work after making sure that it is analyzed by your TA and/or you are given the permission by your TA to upload.

Part 1. Preliminary Work (50 points)

1. ArrayOperations. (20 points)

Define an array in the data segment. Also provide its size in a variable. See the example below. You may use different variable names and array definitions/ The following is just for illustration.

```
array:    .word 2, 4, 5, 2, 5, 0, 4, 0, 0, 4
arrSize:  .word 10
```

Define four subprograms: *PrintArray* , *FindSum* , *FindMinMax* , and *CountAnEntry*.
In all of them use $a0 to pass the address of the array, $a1: array size in number of entries and more argument registers when needed.

- *PrintArray*: Prints the array. The registers $a0 and $a1 are respectively used to pass the array beginning address and array size. Use similar ways of passing arguments to the other subprograms.
- *FindSum*: Finds and returns the summations of all array elements.
- *FindMinMax*: Finds and returns minimum and maximum entry of the array.
- *CountAnEntry*: In this subprogram as the third argument pass an array location and count the number of entries equal to array element at that position of the array. For the array (2, 4, 5, 2, 5, 0, 4, 0, 0, 4), the 2nd array element 4 appears 3 times in the array.

2. FindOccurrences. (30 points)

In this work the subprograms to be written are: *CreateInitArray*, *PrintArray*, and *FindFreq*. See the following for their definitions. You may have additional subprograms as needed.

Main program, the one right after .text, performs the following steps. First it calls CreateInitArray.

The subprogram CreateInitArray

1. Asks the user to enter the size (in words) of the array to be created then dynamically allocates storage using syscall 9.
2. Initializes the array content with positive integers by using a simple user interface.
4. Returns array address and array size to Main (in $v0 and $v1 registers).

Main calls PrintArray to print the contents of the array entered by the user. To do that it passes array address and size to PrintArray.

Main invokes FindFreq. See below for its description.

Main calls PrintArray to print the content of FreqTable generated by FindFreq.

The subprogram FindFreq performs the following.

It receives the address of the created by CreateInitArray and its size, and address of FreqTable defined in the data segment in argument ($a0, $a1, $a2) registers

and finds number of times the numbers 0 to 9 appear in the array entered by the user and stores this frequency information into FreqTable.

FreqTable .word 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 # Initial description in the data segment.

FreqTable 1st word will be used to store the number times the number 0 appears in the array created by the user.

...

FreqTable 10th word contains the number times the number 9 appears in the array

For example, if the array created by the user contains (0, 1, 2, 3, 2, 90, 2, 8, 1, 20); FreqTable will contain 1, 2, 2, 1, 0, 0, 0, 0, 1, 0.

Part 2. Lab Work (50 points)

1. ExchangeRegBytes (30 points)

Write a program that exchanges “first and second bytes” and “third and fourth bytes” of a register.

For example, if the register contains for 0X01 02 03 04 the reversed number is 0X02 01 04 03

You have to use subprograms in your implementation. You have to decide what they are. Provide a reasonable user interface. The user interface would ask the user to continue with another number.

Enter a decimal number and display it as a hexadecimal number using syscal 34. Likewise display the new register contents in hexadecimal.

2. ReverseArray (20 points)

Write a program that reverses the content of an array. For example, if the array contains 10, 20, 30, 40, 50 after running the program it contains 50, 40, 30, 20, 10. Use proper input interface and for reversing the array use a subprogram. To the subprogram pass the array beginning address and the array size. You may define the array in the data segment the dynamic storage allocation using syscall 9 is optional.

Part 3. Submit Lab Work for MOSS Similarity Testing

1. Submit your Lab Work MIPS codes for MOSS similarity- plagiarism testing to Moodle.
2. You will upload one file. Use filename **StudentID_FirstName_LastName_SecNo_LAB_LabNo.txt**
3. Only a NOTEPAD FILE (txt file) is accepted. No txt file upload means you get 0 from the lab. Please note that we have several students and efficiency is important.
4. *Even if you didn’t finish, or didn’t get the MIPS codes working, you must submit your code to the Moodle Assignment for similarity checking.*

Part 4. Cleanup

1. After saving any files that you might want to have in the future to your own storage device, erase all the files you created from the computer in the lab.
2. When applicable put back all the hardware, boards, wires, tools, etc where they came from.
3. Clean up your lab desk, to leave it completely clean and ready for the next group who will come.

LAB POLICIES

1. You can do the lab only in your section. Missing your section time and doing in another day is not allowed.
2. The questions asked by the TA will have an effect on your lab score.
3. Lab score will be reduced to 0 if the code is not submitted for similarity testing, or if it is plagiarized. MOSS-testing will be done, to determine similarity rates. Trivial changes to code will not hide plagiarism from MOSS—the algorithm is quite sophisticated and powerful works for ChatGPT code too. Please also note that obviously you should not use any program available on the web, or in a

book, etc. since MOSS will find it. The use of the ideas we discussed in the classroom is not a problem.

4. You must be in lab, working on the lab, from the time lab starts until your work is finished and you leave.
5. No cell phone usage during lab.
6. Internet usage is permitted only to lab-related technical sites.